

Smart Guard — Part 2 explain

Contents

Part 2 — Code & Architecture Explanation	1
1. Goals (from the course brief)	1
2. System architecture (Part 2)	2
3. File map (Part 2-centric)	2
4. REST routing in C (server.c)	3
5. Telemetry module (telemetry.c / telemetry.h)	5
6. Shared person / history state (persons_state.c)	6
7. FastAPI / Swagger gateway (gateway/main.py)	7
8. Dashboard integration (Part 2 UX)	8
9. Concurrency under Part 2 load tests	9
10. Network disconnect experiment (2-4) vs code	9
11. Build & link notes	9
12. Configuration keys used in Part 2	9
13. End-to-end example: telemetry request	10
14. How Part 2 connects to other parts	10
15. Quick reference commands	10
16. PDF report package (what this part must contain)	10
17. Results analysis (Part 2)	11
18. Problems encountered and how they were solved (Part 2)	11
19. Conclusion	12
20. Related documents	12

Part 2 — Code & Architecture Explanation

Field	Value
Document	Final architecture & code explanation (submission)
Status	Complete — aligned with report/part 2/report.md
Companion	report/part 2/report.md (mandatory experiments 2-1 ... 2-4 + Swagger)
Also covers	PDF package: architecture, code, experiment tables/charts, results analysis, problems & solutions
Project	Smart Guard System
Student	Parsa Nikookalam · 402102657
Scope	REST APIs in C, live telemetry, shared state, Swagger gateway

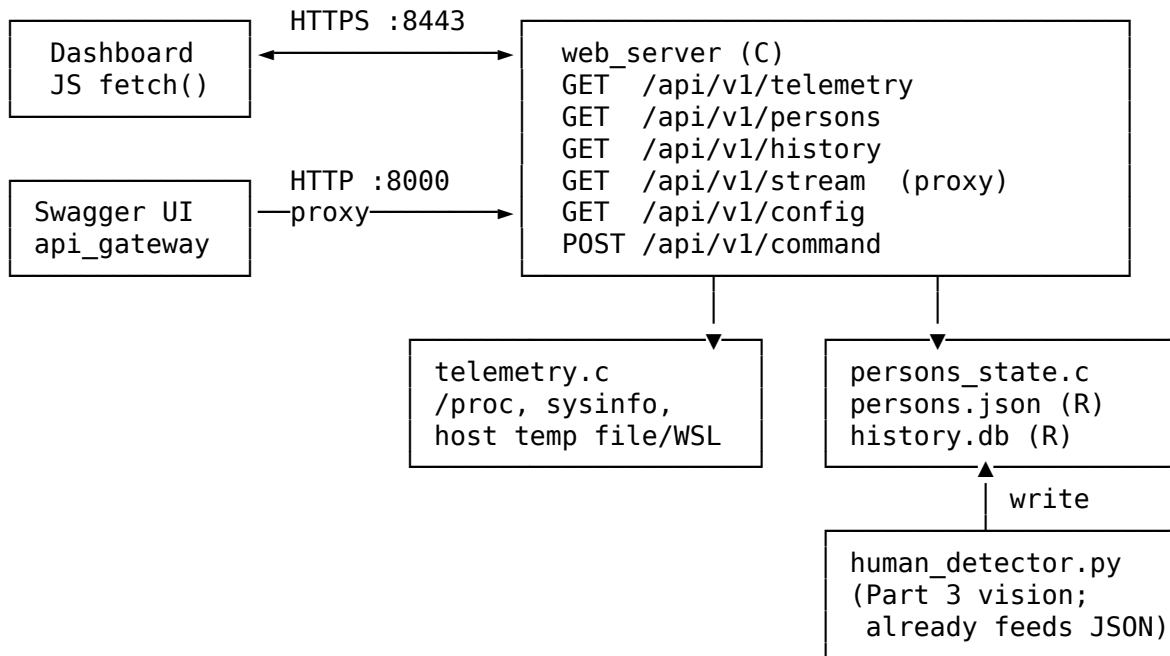
Part 2 does **not** replace the Part 1 server. It **extends** web/src/server.c with JSON endpoints and adds a documentation-only proxy.

1. Goals (from the course brief)

Requirement	Implementation
REST APIs	Handled inside C HTTPS server (server.c)
Live system telemetry	telemetry.c → GET /api/v1/telemetry
Swagger / API docs	gateway/main.py proxies to C; UI at :8000/docs
Core logic still in C	Gateway must not implement business rules

Also used heavily in Part 2 **experiments**: temperature sampling, C process memory, concurrent curl load on telemetry, behaviour under network loss.

2. System architecture (Part 2)



Rule of ownership:

Concern	Owner
HTTP routing, JSON responses, stream proxy, commands	C
Measuring CPU/RAM/temp	C (telemetry.c)
Writing person counts / history rows	Python detector
OpenAPI documentation UI	Python gateway (proxy only)

3. File map (Part 2-centric)

```

web/
├── src/
│   ├── server.c           # Route table + stream proxy + command dispatcher
│   ├── telemetry.c/.h     # cpu_temp, mem, cpu_usage
│   └── persons_state.c/.h # Read persons.json + SQLite history/blackbox
└── gateway/
    └── main.py            # FastAPI + Swagger + HTTP/1.0 command helper
  
```

```

└─ requirements.txt
└─ static/                # Local Swagger UI assets (offline /docs)
services/
└─ web_server.service
└─ api_gateway.service
data/
└─ persons.json           # { "count", "timestamp" }
└─ history.db             # detections table (written by detector)

```

4. REST routing in C (server.c)

4.1 Request intake

After TLS accept and SSL_accept:

1. `ssl_read_http_request()` accumulates bytes until headers + Content-Length body are complete (needed because some clients split TLS records).
2. `path_match(req, "GET", "/api/v1/...")` finds routes (case-tolerant).
3. Handlers call `send_ssl_response()` with JSON or stream bytes.

4.2 Endpoint catalogue (Part 2 core)

Method	Path	Handler behaviour
GET	/api/v1/telemetry	<code>get_system_telemetry()</code> → JSON
GET	/api/v1/persons	<code>read_persons_snapshot()</code> → JSON
GET	/api/v1/history	<code>read_persons_history()</code> last 5 rows → JSON
GET	/api/v1/config	Student name/ID from globals loaded at start
GET/HEAD	/api/v1/stream	<code>handle_stream_proxy()</code> → MJPEG from detector
POST	/api/v1/command	<code>handle_command()</code> parse "cmd"

4.3 Source: REST handlers in server.c

```

if (path_match(buffer, "GET", "/api/v1/telemetry")) {
    SystemTelemetry t;
    get_system_telemetry(&t);
    char json[512];
    snprintf(json, sizeof(json),
        "{\"cpu_temp\": %.2f, \"free_mem_kb\": %ld, "
        "\"mem_used_percent\": %.2f, \"cpu_usage_percent\": %.2f}",
        t.cpu_temp, t.free_mem_kb, t.mem_used_percent, t.cpu_usage_percent);
    send_ssl_response(ssl, 200, "OK", "application/json", json);
    return;
}

if (path_match(buffer, "GET", "/api/v1/persons")) {
    PersonSnapshot snap;
    read_persons_snapshot(&snap);
    char json[256];
    snprintf(json, sizeof(json),
        "{\"count\": %d, \"timestamp\": %ld}", snap.count, snap.timestamp);
    send_ssl_response(ssl, 200, "OK", "application/json", json);
    return;
}

```

```

if (path_match(buffer, "GET", "/api/v1/history")) {
    PersonHistory hist;
    read_persons_history(&hist);
    /* build {"records":[{"count":..., "timestamp":...}, ...]} */
    send_ssl_response(ssl, 200, "OK", "application/json", json);
    return;
}

```

Example telemetry payload:

```

{
    "cpu_temp": 72.50,
    "free_mem_kb": 4123456,
    "mem_used_percent": 48.20,
    "cpu_usage_percent": 23.10
}

```

4.4 Stream proxy (handle_stream_proxy)

Purpose: browser talks **only** to the C HTTPS server; it never needs to know about Python :5000.

Algorithm:

1. TCP connect to STREAM_HOST:STREAM_PORT (default 127.0.0.1:5000).
2. Send GET /video_feed HTTP/1.1.
3. Read upstream response; rewrite outer headers to:

Content-Type: multipart/x-mixed-replace; boundary=frame

4. Pipe body chunks with SSL_write until client or upstream closes.

If the detector is down → 502 JSON detector_not_running.

4.5 Source: MJPEG proxy (abbreviated)

```

static void handle_stream_proxy(SSL *ssl) {
    int upstream = socket(AF_INET, SOCK_STREAM, 0);
    /* connect to STREAM_HOST:STREAM_PORT (default 127.0.0.1:5000) */
    char req[256];
    snprintf(req, sizeof(req),
        "GET /video_feed HTTP/1.1\r\nHost: %s:%d\r\nConnection: close\r\n\r\n",
        g_stream_host, g_stream_port);
    write(upstream, req, strlen(req));

    /* After upstream headers, rewrite for the browser: */
    snprintf(out_hdr, sizeof(out_hdr),
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: multipart/x-mixed-replace; boundary=frame\r\n"
        "Cache-Control: no-cache\r\n"
        "Connection: close\r\n"
        "Access-Control-Allow-Origin: *\r\n"
        "\r\n");
    SSL_write(ssl, out_hdr, hn);
    /* then pipe body chunks: read(upstream) → SSL_write(ssl, ...) */
}

```

This is why Part 2 load tests and Part 3 demos can use a single secure origin.

4.6 Command dispatcher (handle_command)

Parses JSON "cmd" from body (or whole request buffer as fallback).

Part 2-relevant commands:

cmd	Behaviour
reboot	On TARGET=wsl: soft marker file only (no host reboot). On board: fork + reboot
camera_on / camera_off	Persist flag for detector (see Part 3/4)

Later parts add guard_*, watchdog_*, thermal_*, test_email in the **same** function — one extensible command bus.

5. Telemetry module (telemetry.c / telemetry.h)

5.1 Public API

```
typedef struct {  
    float cpu_temp;  
    float mem_used_percent;  
    long free_mem_kb;  
    float cpu_usage_percent;  
} SystemTelemetry;
```

```
int get_system_telemetry(SystemTelemetry *telemetry);
```

5.2 Source: get_system_telemetry + CPU usage (telemetry.c)

```
int get_system_telemetry(SystemTelemetry *t)  
{  
    if (!t)  
        return -1;  
    t->cpu_temp = read_cpu_temp();  
    read_memory_info(&(t->mem_used_percent), &(t->free_mem_kb));  
    t->cpu_usage_percent = read_cpu_usage();  
    return 0;  
}  
  
static float read_cpu_usage(void)  
{  
    static unsigned long long prev_user = 0, prev_nice = 0, /* ... */ prev_steal = 0;  
    FILE *fp = fopen("/proc/stat", "r");  
    /* fscanf cpu user nice system idle iowait irq softirq steal */  
    unsigned long long total_diff = current_total - prev_total;  
    unsigned long long idle_diff = current_idle_total - prev_idle_total;  
    /* save current counters into prev_* */  
    if (total_diff == 0)  
        return 0.0f;  
    return 100.0f * (float)(total_diff - idle_diff) / (float)total_diff;  
}
```

5.3 CPU usage (summary)

1. Read /proc/stat aggregate cpu line (user/nice/system/idle/iowait/...).

2. Compare with **static previous counters** from the last call (no internal sleep).
3. $\text{usage\%} = 100 * (\Delta\text{total} - \Delta\text{idle}) / \Delta\text{total}$.

The first call after process start often returns 0 (no prior sample). Dashboard polls every few seconds, so later samples are meaningful.

5.4 Memory

Uses `sysinfo()` (or `/proc/meminfo` patterns):

- `free_mem_kb`
- $\text{mem_used_percent} = \text{used} / \text{total} * 100$

5.5 Temperature — board vs WSL

Orange Pi / native Linux:

- Scan `/sys/class/thermal/thermal_zone*/temp`
- Scan `/sys/class/hwmon/*/temp*_input`
- Prefer sensors whose names look like CPU/SoC (x86_pkg, cpu-thermal, k10temp, ...) via a scoring function

WSL:

Linux `sysfs` often has **no real package temp**. So:

1. Prefer a **cache file** written by Windows (HostCpuTemp task), e.g. `/mnt/c/Users/<user>/AppData/Local/SmartGuard/cpu_temp.txt`
2. Or `data/host_cpu_temp.txt`
3. Fallback: run `scripts/host_cpu_temp.sh` (PowerShell) with a short in-process cache (PowerShell is slow)

`web_server.service` sets:

```
Environment="HOST_CPU_TEMP_CMD=/bin/bash ../scripts/host_cpu_temp.sh"
Environment="PATH=../mnt/c/Windows/System32/WindowsPowerShell/v1.0"
```

so `systemd` can still reach Windows tools.

5.6 Why this matters for experiments

Part 2 mandatory tests sample `/api/v1/telemetry` every 30 s (temp curves) and under curl load. All of that data comes from this module — **no Python telemetry service**.

6. Shared person / history state (`persons_state.c`)

6.1 `persons.json` (IPC file)

Written by Python detector; read by C:

```
{"count": 1, "timestamp": 1710000000}
```

`read_persons_snapshot()` does a small-string parse (`strstr + sscanf`) — intentionally dependency-light (no full JSON library required in C).

6.2 Source: reading `persons.json` (`persons_state.c`)

```
#define PERSONS_JSON_PATH "../data/persons.json"
```

```
int read_persons_snapshot(PersonSnapshot *out)
```

```

{
    FILE *fp = fopen(PERSONS_JSON_PATH, "r");
    char buf[256] = {0};
    fread(buf, 1, sizeof(buf) - 1, fp);
    fclose(fp);

    int count = 0;
    long ts = 0;
    char *p = strstr(buf, "\"count\"");
    if (p)
        sscanf(p, "\"count\"%[^0-9-]%d", &count);
    p = strstr(buf, "\"timestamp\"");
    if (p)
        sscanf(p, "\"timestamp\"%[^0-9-]%ld", &ts);

    out->count = count;
    out->timestamp = ts;
    return 0;
}

```

Used by:

- GET /api/v1/persons
- Email / MQTT / Part 4 Guard (same snapshot reader)

6.3 history.db (SQLite)

Detector inserts into detections(count, timestamp).

C opens **read-only** and:

- read_persons_history() → last **5** rows for GET /api/v1/history
- read_blackbox_stats() → Part 4 GET /api/v1/blackbox JSON:
{"total_human_events":..., "stored":..., "capacity":500}

This split keeps **writers in Python** (vision cadence) and **readers in C** (API surface).

7. FastAPI / Swagger gateway (gateway/main.py)

7.1 Why a gateway?

The PDF allows Python for Swagger. The gateway:

- Serves OpenAPI docs at http://127.0.0.1:8000/docs
- Proxies REST calls to the C HTTPS server (https://127.0.0.1:8443)
- Uses **local** Swagger static files so /docs works offline
- Does **not** own telemetry/detection logic (C remains the source of truth)

7.2 Demo for the report

Part 2 report includes a short section on this page plus a **screen-recording video**:

- **Watch:** report/part 2/fig/08_swagger_api_demo.mp4
- **Still:** report/part 2/fig/08_swagger_api_demo.png
- Show: open /docs, expand endpoints, **Try it out** on telemetry / persons / command

7.3 Proxy helpers

Helper	Role
<code>_proxy_json</code>	httpx GET/POST to C with verify=False (self-signed)
<code>_c_command_http10</code>	Tiny raw HTTP/1.0 TLS client for POST /command

7.4 Source: HTTP/1.0 command helper (gateway/main.py)

```
def _c_command_http10(cmd: str) -> Tuple[int, bytes, str]:
    """Minimal HTTP/1.0 POST – headers+JSON in one TLS write (Swagger-safe)."""
    body = json.dumps({"cmd": cmd}, separators=(",", ":")).encode("utf-8")
    req = (
        f"POST /api/v1/command HTTP/1.0\r\n"
        f"Host: 127.0.0.1:{HTTPS_PORT}\r\n"
        f"Content-Type: application/json\r\n"
        f"Content-Length: {len(body)}\r\n"
        f"Connection: close\r\n"
        f"\r\n"
    ).encode("ascii") + body

    ctx = ssl._create_unverified_context()
    with socket.create_connection(("127.0.0.1", HTTPS_PORT), timeout=8) as sock:
        with ctx.wrap_socket(sock, server_hostname="127.0.0.1") as ssock:
            ssock.sendall(req)
            # ... read response ...
```

7.5 Why HTTP/1.0 for commands?

Some HTTP/2 or httpx write patterns split headers and body across TLS records. Early C code that did a single `SSL_read` then reported `missing_cmd`.

Fix (two sides):

1. C: `ssl_read_http_request()` loops until Content-Length satisfied.
2. Gateway: send a classic HTTP/1.0 request in **one** `sendall` for commands.

7.6 systemd

`api_gateway.service` runs the module directly (uvicorn is started from `if __name__ == "__main__"` in `gateway/main.py`):

```
WorkingDirectory=/home/parsa/embedded_project/gateway
ExecStart=/home/parsa/embedded_project/.venv/bin/python \
/home/parsa/embedded_project/gateway/main.py
```

That calls `uvicorn.run(app, host="0.0.0.0", port=GATEWAY_PORT)` with `GATEWAY_PORT` from `config.env` (default 8000).

8. Dashboard integration (Part 2 UX)

`index.html` JavaScript:

1. Polls `/api/v1/telemetry` every few seconds → fill CPU/temp/RAM cards.
2. Polls `/api/v1/persons` → person count widget.
3. Sets `` for live video.
4. Uses `/api/v1/config` for name/ID in the header.

All of those requests hit the **same origin** as the page (:8443), so there is no mixed-content problem.

9. Concurrency under Part 2 load tests

Experiment **2-3** fires ~50 parallel GET /api/v1/telemetry requests.

Because each connection is a pthread:

- Accept loop stays responsive.
- Telemetry reads are mostly lock-free local /proc + cached temp.
- Latency rises under TLS handshake + thread scheduling — that rise is what the report measures.

Memory experiment **2-2** samples **RSS of web_server** (/proc/<pid>/status VmRSS) during a long stream — the long-lived stream thread holds the MJPEG proxy open; RSS should stabilize if there is no leak.

10. Network disconnect experiment (2-4) vs code

TA clarification on WSL: disconnect the **WSL ↔ Windows** virtual NIC (ip link set eth0 down / vEthernet (WSL)), while the stream is open from the Windows browser.

Helper: scripts/demo_part2_network_disconnect.sh (2 minutes down, then up; writes session + journal dumps under report/part 2/fig/).

During outage:

- Windows client loses reachability to WSL-forwarded ports → stream stalls
- Inside WSL, loopback 127.0.0.1 and systemd units stay up

Recovery: bring the NIC up again; refresh the browser — no redeploy required.

Component	If uplink dies
C server + detector on same machine	Keep running
Remote browser / MQTT “PC” role	Stalls until link returns
systemd units	Stay active

Recovery is “reconnect TCP/TLS clients”; no special reconnect state machine is required in C for localhost demos.

11. Build & link notes

Part 2 needs:

-lssl -lcrypto -lsqlite3 -lpthread

SQLite is for history reads. Telemetry itself needs no extra library beyond libc.

12. Configuration keys used in Part 2

Key	Consumer
HTTPS_PORT / HTTP_PORT	C listeners
STREAM_HOST / STREAM_PORT	Stream proxy upstream

Key	Consumer
GATEWAY_PORT	Uvicorn
HOST_CPU_TEMP_FILE / helper script	WSL temperature
TARGET	reboot command behaviour

13. End-to-end example: telemetry request

1. Client: GET /api/v1/telemetry over TLS.
2. server.c matches path.
3. get_system_telemetry(&t) fills struct.
4. snprintf JSON → send_ssl_response(200, ...).
5. Dashboard chart or Part 2 CSV logger records cpu_temp.

14. How Part 2 connects to other parts

Part	Relationship
Part 1	Same TLS server; Part 2 adds routes
Part 3	Detector fills persons.json / DB that Part 2 already exposes
Part 4	Extra GET routes (/guard, /blackbox, ...) and more cmd values

15. Quick reference commands

```
curl -sk https://127.0.0.1:8443/api/v1/telemetry
curl -sk https://127.0.0.1:8443/api/v1/persons
curl -sk https://127.0.0.1:8443/api/v1/history
# Swagger
xdg-open http://127.0.0.1:8000/docs # or browser on Windows host
```

16. PDF report package (what this part must contain)

Per the course PDF, Part 2 must include **architecture**, **code explanation**, **all experiment tables and charts**, **results analysis**, and **problems & solutions**.

PDF expectation	Where it lives
Architecture	Sections 2-3 of this explain.md
Code explanation	Sections 4-10 (REST, telemetry, gateway) with source excerpts
Experiment tables & graphs	report/part 2/report.md + fig/
Results analysis	Section 17 below
Problems & solutions	Section 18 below

16.1 Mandatory experiments — tables & figures checklist

No.	Experiment (PDF)	Required tables / charts
2-1	Temp idle / stream / stream+detect (30 s, 5 min)	3-curve temp-vs-time graph; max-temp table
2-2	C memory during 5 min stream (every 5 s)	Memory-vs-time graph; leak analysis
2-3	50 concurrent curls to /api/v1/telemetry	Tables: Δ temp/CPU/mem; latency mean/max/in
2-4	Network disconnect 2 min during stream	Behaviour + logs + recovery
Swagger	OpenAPI docs via gateway	Short explanation + demo video + still of /doc

16.2 Measured result tables (see also report.md)

2-1 — Max temperature (°C)

State	Max temp (°C)
(a) Idle	57.85
(b) Stream only	61.85
(c) Stream + detection	64.85

2-2 — Memory (RSS of web_server)

Metric	Value
Initial / final / peak RSS (KB)	17512 / 17512 / 17512
Leak?	No — flat RSS for full 5-minute window

2-3 — Load vs baseline

Metric	Before	After burst	Δ
cpu_temp	58.85	58.85	0.00
cpu_usage_percent	2.68	4.84	+2.16
Latency mean (s)	—	0.0290	~+9 ms vs min

17. Results analysis (Part 2)

Experiment	What the data showed
2-1	Max temps idle 57.85 < stream 61.85 < detect 64.85 °C
2-2	RSS constant 17512 KB
2-3	CPU +2.16%; latency mean ~29 ms (max ~42 ms); temp flat
2-4	eth0 down 120 s: loopback telemetry OK; journal STREAM_NETWORK_DOWN + [network] ... DISCOM
Swagger	/docs lists C APIs; demo video + still executed Try-it-out flows

18. Problems encountered and how they were solved (Part 2)

#	Problem	Severity	Cause
1	Swagger POST /command → missing_cmd	High	httpx/TLS split body; early sin

#	Problem	Severity	Cause
2	CPU temperature –1 / wrong on WSL	High	No reliable sysfs package tem
3	Stream 502 detector_not_running	Medium	Detector down or port misma
4	Concurrent load spikes latency	Expected	Many TLS handshakes
5	Confusing system RAM vs C RSS	Medium	PDF asks for C program men
6	Turning Windows Wi-Fi OFF did not show a real outage	High (2-4)	Loopback / localhost forwardi
7	Journal had no clear “stream disconnected” line	High (2-4)	TCP hang; no uplink-aware lo
8	Plot script failed with system python3	Medium (2-1)	Matplotlib only in project ven
9	Temperature CSVs empty / one curve only	Medium (2-1)	Sampler not run for all three
10	Demo evidence hard to screenshot from noisy journalctl	Low (2-4)	Long mixed unit logs

18.1 Focus — Experiment 2-4 debugging (detail)

1. Wrong disconnect method

Wi-Fi OFF left `https://127.0.0.1:8443` working, so the experiment looked like a no-op.

Fix: follow TA guidance — bring **eth0 down** inside WSL (`scripts/demo_part2_network_disconnect.sh`).

2. Missing journal evidence for stream loss

Services stayed active (correct), but logs did not state that remote stream clients disconnect.

Fix: uplink monitor + stream proxy disconnect messages; also logger markers `STREAM_NETWORK_DOWN` / `UP` every 30 s while down.

3. Evidence packaging

Long mixed `journalctl` output is hard to screenshot.

Fix: each demo run refreshes focused text evidence:

- `report/part 2/fig/06_network_disconnect_session.log`
 - `report/part 2/fig/06_network_disconnect_journal.txt`
- Plus figures `06_network_disconnect_logs.png` and `07_network_recovery.png`.

19. Conclusion

Part 2 turns the Part 1 HTTPS appliance into a **telemetry- and control-capable REST device**:

Layer	Responsibility
C (server.c, telemetry.c, persons_state.c)	Measurement, JSON APIs, stream proxy, commands, uplink/stream
Shared files / SQLite	Bridge from vision (writer) to API (reader)
FastAPI gateway	OpenAPI /docs + transparent proxy only (08_swagger_api_de
Scripts	Temp sampling, Part 2 plots (.venv), WSL↔Windows disconnec

Measured highlights: detect state hottest (**64.85 °C**); RSS **flat 17512 KB** (no leak); burst latency mean **~29 ms**; `eth0 down` **120 s** with clear journal disconnect markers. Core rule held: **logic in C**; Python limited to vision + thin docs/helpers.

All mandatory experiments **2-1 ... 2-4** and the Swagger demo are complete.

20. Related documents

Document	Role
<code>report/part 2/report.md</code>	Mandatory experiments 2-1 ... 2-4 (final)

Document	Role
report/part 2/explain.md	This file — architecture, code, analysis, problems
report/part 1/explain.md	HTTPS server foundation
README.md (repo root)	Full-project setup guide